
Engineering Documentation: AI/ML Phase 1, Week 1

Document Status: Approved & Completed

Author: Jainish Mehta

Focus: Environment Configuration, Version Control, Dependency Isolation, and OOP API Consumption.

Executive Summary

The objective of Week 1 was to transition from a beginner-level coding mindset to a production-grade software engineering workflow. Before writing machine learning algorithms, a robust, scalable, and isolated environment must be established. This week focused on mastering the Command Line Interface (CLI), Version Control (Git/GitHub), Virtual Environments (venv), and Object-Oriented Programming (OOP) in Python.

Technology Stack

- **IDE:** Visual Studio Code (VS Code)
- **Terminal:** Git Bash (Linux-based CLI environment)
- **Version Control:** Git & GitHub
- **Language:** Python 3.x
- **Libraries:** `requests`, `datetime`

17 July Daily Execution Log

Day 1 & 2: Environment Configuration & Version Control

- **Objective:** Set up the IDE, configure Git, and establish a connection to a remote GitHub repository.
- **What was done:**
 - Installed VS Code with Python and GitLens extensions.
 - Configured global Git user variables (`user.name` , `user.email`).

- Replaced the default Windows PowerShell terminal with **Git Bash** to ensure compatibility with Linux commands (`pwd` , `ls` , `cd` , `mkdir`).
- Initialized a local Git repository (`git init`) and connected it to a remote GitHub repository via SSH/HTTPS.
- **Why we did it (The Rationale):**
 - AI models are deployed on Linux-based cloud servers (AWS, GCP). Relying on Windows UI or PowerShell creates a knowledge gap. Mastering Git Bash ensures seamless cloud deployment later. Furthermore, Git is mandatory for team collaboration and code versioning.

Day 3 & 4: Project Architecture & Dependency Management

- **Objective:** Implement standard Machine Learning folder structures and resolve "Dependency Hell."
- **What was done:**
 - Constructed a hybrid directory structure integrating the "Cookiecutter Data Science" standard (`src/` , `data/` , `models/` , `notebooks/`).
 - Utilized `.gitkeep` files to force Git to track empty architectural folders.
 - Authored a robust `.gitignore` file to prevent the upload of massive datasets and virtual environments.
 - Initialized and activated a Python Virtual Environment (`python -m venv venv`).
 - Installed the `requests` library and generated a `requirements.txt` file (`pip freeze`).
- **Why we did it (The Rationale):**
 - Beginners install packages globally, which leads to version conflicts across different projects. Virtual Environments isolate dependencies. The `.gitignore` file acts as a security shield, preventing GitHub repositories from crashing due to the 100MB file limit on large ML datasets.

Day 5 & 6: Object-Oriented Programming (OOP) & API Integration

- **Objective:** Move away from procedural scripting and build modular, class-based Python code.
- **What was done:**

- Configured the VS Code Python Interpreter path to point to the local `venv` , resolving global path warnings.
- Developed `scraper.py` , utilizing Python `Classes` and the `__init__` constructor.
- Built methods utilizing the `requests` library to fetch JSON data from a public RESTful API (Joke API) and parse the payload.
- **Why we did it (The Rationale):**
 - Modern Deep Learning frameworks (like PyTorch) are entirely built on OOP principles. An AI Engineer must intimately understand `Classes` , `self` , and inheritance to build and debug neural networks.

Day 7: Iteration, Error Handling, & File I/O (Mini-Project)

- **Objective:** Build a functional, robust mini-project mimicking real-world data extraction.
- **What was done:**
 - Developed `crypto_tracker.py` , an OOP-based script to fetch live cryptocurrency prices via the CoinGecko API.
 - Implemented `try/except` blocks for professional error handling (preventing script crashes during API timeouts).
 - Implemented File I/O (with `open("file.txt", "a")`) to append live price data, accompanied by system timestamps (`datetime`), into a local log file.
 - Updated `.gitignore` to exclude the newly created local dataset folder (`saved_data/`).
- **Why we did it (The Rationale):**
 - Machine Learning Engineers spend 80% of their time extracting and cleaning data. This project simulated a live data pipeline. Implementing `append` mode rather than `write` mode ensures historical data is preserved, which is critical for time-series AI forecasting.

Key Deliverables Achieved

1. **Professional Repository:** A fully structured, version-controlled GitHub repository.
2. **Environment Isolation:** Mastery over `venv` and `requirements.txt` .
3. **Data Extraction Pipeline:** A functional, error-handled OOP Crypto Tracker that successfully requests remote API data and writes to local storage without exposing

datasets to the cloud.

Next Steps (Week 2 Preview)

With the software engineering foundation secured, the focus shifts to **Data Engineering**. Week 2 will cover SQL, advanced Pandas data manipulation, and extracting structured data from relational databases.